Chapter 1

Web Server scripting language in PHP

OutLine

- Introduction to Server-Side Programming – PHP
- Basic PHP
- OOP Concept
- Session and cookies
- Exception handling

Server Side Scripting Basics

- **Script:** A program/code/ instruction/command
- **Server side:** The control of the script is handled by the web crossing server.
- **Server side scripting:** writing a program that is executed on the server

When a server side script executes :

The server runs the script and send standard HTML page to each user

Architecture

Three tire architecture

User/client------→web server-------→ Database

Server side Scripting languages

- ASP/active server page
- JSP/Java Server Page
- Pascal
- PHP
- Python

PHP

- An acronym for Hyper Text Pre Processor
- A server side scripting language
- Used to manage dynamic contents, Databases and Session Tracking
- It is integrated with a # of Data bases like MySQL, Oracle, Informix, MS SQL,….

PHP cont'd

- PHP syntax is C-like
- IT is Open source

Why to choose PHP

- It can run on different platforms
- Compatible with most servers
- Run efficiently on the server
- It is free to download

PHP environment

In order to develop and run PHP Web pages, three vital components need to be installed on your computer system.

Web Server:

- PHP will work with virtually all Web Server software, but the most often used is freely available Apache Server.
- Download Apache for free here: *http://httpd.apache.org/download.cgi*

PHP Environment

- **Database:**
  - PHP will work with virtually all database software, but most commonly used is freely available MySQL database.
  - Download MySQL for free here:
    - *http://www.mysql.com/downloads/index.html*

PHP environment

PHP Parser - In order to process PHP script instructions, a parser must be installed to generate HTML output that can be sent to the Web Browser.

PHP environment setup

WAMP/XAMP

- An integrated tool (Apache+MySQL+PHP)
- You can use wamp as a parser, database, server.
- Download wamp from internet
- Using PhpMyAdmin of wamp we can
    - Manage Databases
    - Manage tables
    - Import/export data
- For our class we will use this tool i.e
    - Server->Apache
    - Database-> MySQL
    - SSL->PHP
- Our server name will be *locallhost*

PHP Basics

PHP File

- PHP file have extension of '.php', '.php3', '.phtml'
- The default file extension for PHP files is ".php".
- May contain Texts, HTML tags and Scripts
- If you are using WAMP
- you have to save all PHP files to www directory (c:/wamp/www/)
- But here U better have your own folder(name of ur project)
- In order to run ur PHP script U need to write the address of the file on the browsers' address bar
- E.g. *http://locallhost/myproject/myfile.php*

PHP Basics

PHP tags/Basic PHP Syntax

- **In a PHP file U have to enclose php scripts for the parser to differentiate them.**
- A PHP script can be placed anywhere in the document.
- **We can use four different ways to do this**
  - **Canonical PHP tags:**
    - **<?php.....?>**
    - **the most common and effective tag**
  - **Short tags:**
    - **<?.....?>**
    - **But to use this tag U need to have some configuration**
  - **ASP-style tags:**
    - **<%.....%>**
    - **Again U need configuration in php.ini file**

PHP Basics

PHP tags

- HTML script tags:
  - <script language="PHP">.....</script>
- PHP statements end with a semicolon (;).

PHP Basics

PHP Comments

- A comment in PHP code is a line that is not executed as a part of the program. Its only purpose is to be read by someone who is looking at the code.
- Comments can be used to:
  - Let others understand your code
  - Remind yourself of what you did - Most programmers have experienced coming back to their own work a year or two later and having to re-figure out what they did. Comments can remind you of what you were thinking when you wrote the code
- PHP supports several ways of comments:

```
<?php
# This is single a comment, and
// This is also single a comment.
/* but this is a
  multi line Comment        */
?>
```

PHP Output Statements

- With PHP, there are two basic ways to get output: echo and print
- echo and print are more or less the same. They are both used to output data to the screen.
- The differences are small:
  - echo has no return value while print has a return value of 1 so it can be used in expressions.
  - echo can take multiple parameters (although such usage is rare) while print can take one argument.
  - echo is marginally faster than print.
- **The PHP echo Statement**
- The echo statement can be used with or without parentheses: echo or echo().

The following example shows how to output text with the echo command (notice that the text can contain HTML markup):

Example

```
<?php
echo "<h2>PHP is Fun!</h2>";
echo "Hello world!<br>";
echo "I'm about to learn PHP!<br>";
echo "This ", "string ", "was ", "made ", "with multiple parameters.";
?>
```

PHP Output Statements

- The following example shows how to output text and variables with the echo statement:

Example

```
<?php
$txt1 = "Learn PHP";
$txt2 = "W3Schools.com";
$x = 5;
$y = 4;

echo "<h2>" . $txt1 . "</h2>";
echo "Study PHP at " . $txt2 . "<br>";
echo $x + $y;
?>
```

- **The PHP print Statement**
- The print statement can be used with or without parentheses: print or print()

PHP Output Statements

- The following example shows how to output text with the print command (notice that the text can contain HTML markup):
- Example

```
<?php
print "<h2>PHP is Fun!</h2>";
print "Hello world!<br>";
print "I'm about to learn PHP!";
?>
```

- The following example shows how to output text and variables with the print statement:

Example

```
<?php
$txt1 = "Learn PHP";
$txt2 = "W3Schools.com";
$x = 5;
$y = 4;
print "<h2>" . $txt1 . "</h2>";
print "Study PHP at " . $txt2 . "<br>";
print $x + $y;
?>
```

PHP variables

- Variables are "containers" for storing information.
- All variables in PHP start with a $ symbol.
- Variables can, but do not need, to be declared before assignment.
- Variable names must begin with a letter or underscore character.
- A variable name can consist of numbers, letters, underscores but you cannot use characters like + , - , % , (,) . & , ….
- Remember that PHP variable names are case-sensitive!
- E.g.
- $_name;
- $Father_Name;
- $x=5;
- $y=6;
- $div=$x/$y;
- $sex="Male";
- $conn=mysqli_connect("host","user","pssword");

PHP Variable Types

- PHP has a total of **eight data types** which we use to construct our variables:
  - **Integers:** are whole numbers, without a decimal point, like
  - **Doubles:** are floating-point numbers,
  - **Booleans**: have only two possible values either TRUE or FALSE.
  - **NULL**: is a special type that only has one value: NULL.
  - **Strings**: are sequences of characters
  - **Arrays**: are named and indexed collections of other values.
  - **Objects**: are instances of programmer-defined classes
  - **Resources**: are special variables that hold references

PHP Variables

String Functions

- **strtolower($String);** change the case of a string to lowercase
- **strtoupper($String);** change the case of a string to uppercase
- **ucfirst($String);** make the first letter uppercase
- **.trim($String);** to concatenate two strings without space between them.
- **str_replace("$String1", "$String2", $BigString); to replace *String1* with *String2* in the string *BigString***

e.g. String Functions

```
<?php
$firstString = "The quick brown fox";
$secondString = " jumped over the lazy dog.";
// Concatentation
$thirdString = $firstString;
$thirdString .= $secondString;
echo $thirdString; ?>
Lowercase: <?php echo strtolower($thirdString); ?> <br />
Uppercase: <?php echo strtoupper($thirdString); ?> <br />
Uppercase first-letter: <?php echo ucfirst($thirdString); ?> <br />
Uppercase words: <?php echo ucwords($thirdString); ?> <br />
Length: <?php echo strlen($thirdString); ?> <br />
Trim: <?php echo $fourthString = $firstString . trim($secondString); ?> <br />
Replace by string: <?php echo str_replace("quick", "super-fast",
    $thirdString); ?> <br />
```

PHP Introduction

Type Casting

- PHP will try to convert between types (strings, number, arrays, etc.) as best it can
- **gettype();** will retrieve an items type
  - gettype($var1);
- **Settype();** will convert an item to a specific type
  - settype($var1,"String");
- You can also specify the new type in parentheses in front of the item
  - $var3 = (int) $var1;
  - echo gettype($var3);

e.g. type casting

```
<?php
$var1 = "2 brown foxes";
$var2 = $var1 + 3;
echo $var2;
?>
<br />
<?php
// gettype will retrieve an item's type
echo gettype($var1); echo "<br />";
echo gettype($var2); echo "<br />";
// settype will convert an item to a specific type
settype($var2, "string");
echo gettype($var2); echo "<br />";
// Or you can specify the new type in parentheses in front of the item
$var3 = (int) $var1;
echo gettype($var3); echo "<br />";
?>
```

PHP Introduction

Constants

- **A constant** is an identifier (name) for a simple value.
- The value cannot be changed during the script.
- A valid constant name have no $ sign before the name
- To set a constant, use the define() function
- it takes three parameters:
  - The first parameter defines the ***name*** of the constant,
  - The second parameter defines the ***value*** of the constant, and
  - The optional third parameter specifies whether the constant name should be *case-insensitive*. Default is false.

Cont'd

```
<?php
   define("GREETING", "Welcome to PHP
   World!");
   echo GREETING;
   ?>
```

- The above program creates a **case-sensitive constant**

```
<?php
   define("GREETING", "Welcome to PHP
   World!", true);
   echo greeting;
   ?>
```

- The above program creates a **case-insensitive constant**

PHP Operators

- Operators are used to perform operations on variables and values.
- PHP divides the operators in the following groups:
  - Arithmetic operators
  - Assignment operators
  - Comparison operators
  - Increment/Decrement operators
  - Logical operators
  - String operators
  - Array operators
  - Conditional assignment operators

PHP Operators

- **PHP Arithmetic Operators**
- The PHP arithmetic operators are used with numeric values to perform common arithmetical operations, such as addition, subtraction, multiplication etc.

Operator	Name	Example	Result
+	Addition	$x + $y	Sum of $x and $y
-	Subtraction	$x - $y	Difference of $x and $y
*	Multiplication	$x * $y	Product of $x and $y
/	Division	$x / $y	Quotient of $x and $y
%	Modulus	$x % $y	Remainder of $x divided by $y
**	Exponentiation	$x ** $y	Result of raising $x to the $y'th

PHP Operators

- **PHP Assignment Operators**
- The PHP assignment operators are used with numeric values to write a value to a variable.
- The basic assignment operator in PHP is "=". It means that the left operand gets set to the value of the assignment expression on the right.

Assignment	Same as...	Description
x = y	x = y	The left operand gets set to the value of the expression on the right
x += y	x = x + y	Addition
x -= y	x = x - y	Subtraction
x *= y	x = x * y	Multiplication
x /= y	x = x / y	Division
x %= y	x = x % y	Modulus

PHP Operators

- **PHP Comparison Operators**

The PHP comparison operators are used to compare two values (number or string):

Operator	Name	Example	Result
==	Equal	$x == $y	Returns true if $x is equal to $y
===	Identical	$x === $y	Returns true if $x is equal to $y, and they are of the same type
!=	Not equal	$x != $y	Returns true if $x is not equal to $y
<>	Not equal	$x <> $y	Returns true if $x is not equal to $y
!==	Not identical	$x !== $y	Returns true if $x is not equal to $y, or they are not of the same type
>	Greater than	$x > $y	Returns true if $x is greater than $y
<	Less than	$x < $y	Returns true if $x is less than $y
>=	Greater than or equal to	$x >= $y	Returns true if $x is greater than or equal to$y
<=	Less than or equal to	$x <= $y	Returns true if $x is less than or equal to $y
<=>	Spaceship	$x <=> $y	Returns an integer less than, equal to, or greater than zero, depending on if $x is less than, equal to, or greater than $y.

PHP Operators

- **PHP Logical Operators**

The PHP logical operators are used to combine conditional statements.

Operator	Name	Example	Result
and	And	$x and $y	True if both $x and $y are true
or	Or	$x or $y	True if either $x or $y is true
xor	Xor	$x xor $y	True if either $x or $y is true, but not both
&&	And	$x && $y	True if both $x and $y are true
||	Or	$x || $y	True if either $x or $y is true
!	Not	!$x	True if $x is not true

PHP Operators

- **PHP String Operators**

PHP has two operators that are specially designed for strings.

Operator	Name	Example	Result
.	Concatenation	$txt1 . $txt2	Concatenation of $txt1 and $txt2
.=	Concatenation assignment	$txt1 .= $txt2	Appends $txt2 to $txt1

PHP Conditional Statements

- Conditional statements are used to perform different actions based on different conditions.
- **PHP - The if Statement**
- The if statement executes some code if one condition is true.
- Syntax

```
if (condition) {
   //code to be executed if condition is true;
}
```

Example

```
<?php
$t = date("H");
if ($t < "20") {
    echo "Have a good day!";
}
?>
```

The above code display Output "Have a good day!" if the current time (HOUR) is less than 20:

PHP Conditional Statements

- **PHP - The if...else Statement**

The if...else statement executes some code if a condition is true and another code if that condition is false.

Syntax

```
if (condition) {
    code to be executed if condition is true;
} else {
    code to be executed if condition is false;
}
```

Example

Output "Have a good day!" if the current time is less than 20, and "Have a good night!" otherwise:

```
<?php
$t = date("H");
if ($t < "20") {
    echo "Have a good day!";
} else {
    echo "Have a good night!";
}
?>
```

PHP Conditional Statements

- **PHP - The if...elseif...else Statement**
- The if...elseif...else statement executes different codes for more than two conditions.
- Syntax

```
if (condition) {
   #code to be executed if this condition is true;
} elseif (condition) {
   #code to be executed if first condition is false and this condition
is true;
}
 else {
   #code to be executed if all conditions are false;
}
```

PHP Conditional Statements

Example

Output "Have a good morning!" if the current time is less than 10, and "Have a good day!" if the current time is less than 20. Otherwise it will output "Have a good night!":

```
<?php
$t = date("H");
if ($t < "10") {
    echo "Have a good morning!";
} elseif ($t < "20") {
    echo "Have a good day!";
} else {
    echo "Have a good night!";
}
?>
```

PHP Conditional Statements

- **PHP switch Statement**

Use the switch statement to select one of many blocks of code to be executed.

Syntax

```
switch (n) {
    case label1:
        code to be executed if n=label1;
        break;
    case label2:
        code to be executed if n=label2;
        break;
    case label3:
        code to be executed if n=label3;
      break;
   ...
   default:
      code to be executed if n is different from all labels;
}
```

PHP Conditional Statements

Example

```
<?php
$favcolor = "red";
switch ($favcolor) {
    case "red":
        echo "Your favorite color is red!";
        break;
    case "blue":
        echo "Your favorite color is blue!";
        break;
    case "green":
        echo "Your favorite color is green!";
        break;
    default:
        echo "Your favorite color is neither red, blue, nor green!";
}
?>
```

PHP Loops

- Loops are used to execute the same block of code again and again, as long as a certain condition is true.

In PHP, we have the following loop types:

- The for loop - Loops through a block of code a specified number of times.
- The for loop is used when you know in advance how many times the script should run.
- Syntax

```
for (init counter; test counter; increment counter) {
    //code to be executed for each iteration;
}
```

Examples

The example below displays the numbers from 0 to 10:

```
<?php
 echo " Output " . " <br> ";
for ($x = 0; $x <= 10; $x++) {
    echo "The number is: $x <br>";
}
```

Output
The number is: 0
The number is: 1
The number is: 2
The number is: 3
The number is: 4
The number is: 5
The number is: 6
The number is: 7
The number is: 8
The number is: 9

PHP Loops

PHP while Loop

- The while loop - Loops through a block of code as long as the specified condition is true.

Syntax

```
while (condition is true) {
    code to be executed;
}
```

Examples

The example below displays the numbers from 1 to 5:

```
<?php
$x = 1;
while($x <= 5) {
    echo "The number is: $x <br>";
    $x++;
}
?>
```

Output

The number is: 1
The number is: 2
The number is: 3
The number is: 4
The number is: 5

PHP Loops

PHP do while Loop

- The do...while loop - Loops through a block of code once, and then repeats the loop as long as the specified condition is true.

Syntax

```
do {
    code to be executed;
} while (condition is true);
```

Example

```
<?php
$x = 1;
do {
    echo "The number is: $x <br>";
    $x++;
} while ($x <= 5);
?>
```

Output

The number is: 1
The number is: 2
The number is: 3
The number is: 4
The number is: 5

- Note: In a do...while loop the condition is tested AFTER executing the statements within the loop. This means that the do...while loop will execute its statements at least once, even if the condition is false.

PHP Loops

PHP foreach Loop

- The foreach loop - Loops through a block of code for each element in an array. The foreach loop works only on arrays, and is used to loop through each key/value pair in an array.
- Syntax

```
foreach ($array as $value) {
 // code to be executed;
}
```

Examples

The following example will output the values of the given array ($colors):

```
<?php
$colors = array("red", "green", "blue", "yellow");
foreach ($colors as $value) {
  echo "$value <br>";
}
?>
```

Output

red
green
blue
yellow

PHP Loops

PHP foreach Loop

- The following example will output both the keys and the values of the given array ($age):

Example

```
<?php
$age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");

foreach($age as $x => $val) {
  echo "$x = $val<br>";
}
?>
```

output

Peter = 35
Ben = 37
Joe = 43

PHP Arrays

- An array stores multiple values in one single variable:
- An array is a special variable, which can hold more than one value at a time.
- In PHP, the array() function is used to create an array: array();
- The count() function is used to return the length (the number of elements) of an array:
- In PHP, there are three types of arrays:
  1. Indexed arrays - Arrays with a numeric index
  2. Associative arrays - Arrays with named keys
  3. Multidimensional arrays - Arrays containing one or more arrays

1. PHP Indexed Arrays

- There are two ways to create indexed arrays:
- The index can be assigned automatically (index always starts at 0), like this:

```
$cars = array("Volvo", "BMW", "Toyota");
```

- or the index can be assigned manually:

```
$cars[0] = "Volvo";
$cars[1] = "BMW";
$cars[2] = "Toyota";
```

PHP Arrays

- Example

output

I like Volvo, BMW and Toyota.

```
<?php
$cars = array("Volvo", "BMW", "Toyota");
echo "I like " . $cars[0] . ", " . $cars[1] . " and " . $cars[2] . ".";
?>
```

2. PHP Associative Arrays

- Associative arrays are arrays that use named keys that you assign to them.
- There are two ways to create an associative array:

```
$age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");
```

or:

```
$age['Peter'] = "35";
$age['Ben'] = "37";
$age['Joe'] = "43";
```

- The named keys can then be used in a script:

Example

output

Peter is 35 years old.

```
<?php
$age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");
echo "Peter is " . $age['Peter'] . " years old.";
?>
```

PHP Arrays

- To loop through and print all the values of an associative array, you could use a foreach loop, like this:

Example

```
<?php
$age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");

foreach($age as $x => $x_value) {
    echo "Key=" . $x . ", Value=" . $x_value;
    echo "<br>";
}
?>
```

output

```
Key=Peter, Value=35
Key=Ben, Value=37
Key=Joe, Value=43
```

PHP Arrays

3. PHP Multidimensional Arrays

- A multidimensional array is an array containing one or more arrays.
- PHP supports multidimensional arrays that are two, three, four, five, or more levels deep. However, arrays more than three levels deep are hard to manage for most people.
- **PHP - Two-dimensional Arrays**
- A two-dimensional array is an array of arrays (a three-dimensional array is an array of arrays of arrays).

First, take a look at the following table:

Name	Stock	Sold
Volvo	22	18
BMW	15	13
Saab	5	2
Land Rover	17	15

PHP Arrays

- We can store the data from the table above in a two-dimensional array, like this:

```
$cars = array
  (
  array("Volvo",22,18),
  array("BMW",15,13),
  array("Saab",5,2),
  array("Land Rover",17,15)
  );
```

Now the two-dimensional $cars array contains four arrays, and it has two indices: row and column.

To get access to the elements of the $cars array we must point to the two indices (row and column):Example

```
<?php
echo $cars[0][0].": In stock: ".$cars[0][1].", sold: ".$cars[0][2].".<br>";
echo $cars[1][0].": In stock: ".$cars[1][1].", sold: ".$cars[1][2].".<br>";
echo $cars[2][0].": In stock: ".$cars[2][1].", sold: ".$cars[2][2].".<br>";
echo $cars[3][0].": In stock: ".$cars[3][1].", sold: ".$cars[3][2].".<br>";
?>
```

PHP Arrays

- We can also put a for loop inside another for loop to get the elements of the $cars array (we still have to point to the two indices):

Example

```
<?php
for ($row = 0; $row < 4; $row++) {
  echo "<p><b>Row number $row</b></p>";
  echo "<ul>";
  for ($col = 0; $col < 3; $col++) {
    echo "<li>".$cars[$row][$col]."</li>";
  }
  echo "</ul>";
}
?>
```

output

Row number 0

- Volvo
- 22
- 18

Row number 1

- BMW
- 15
- 13

Row number 2

- Saab
- 5
- 2

Row number 3

- Land Rover
- 17
- 15

Sorting Arrays

- The following are PHP array sort functions:
  - sort() - sort arrays in ascending order
  - rsort() - sort arrays in descending order
  - asort() - sort associative arrays in ascending order, according to the value
  - ksort() - sort associative arrays in ascending order, according to the key
  - arsort() - sort associative arrays in descending order, according to the value
  - krsort() - sort associative arrays in descending order, according to the key

Sorting Arrays

- One of the many advantages arrays have over the other variable types is the ability to sort them.
- PHP includes several functions you can use for sorting arrays, all simple in syntax:
  - $names = array (‘Nohom ', ‘Alemu',‘Kebede');
  - sort($names);
- The sorting functions perform three kinds of sorts.
  - First, you can sort an array by value, discarding the original keys, using sort( ).
  - Second, you can sort an array by value while maintaining the keys, using asort( ).
  - Third, you can sort an array by key, using ksort( ).
- Each of these can sort in reverse order if you change them to rsort( ), arsort( ), and krsort( ) respectively.

PHP Functions

- The real power of PHP comes from its functions.
- PHP has more than 1000 built-in functions, and in addition you can create your own custom functions.
- There are two types of functions:- these are built in and user defined functions

1. PHP Built-in Functions

- PHP has over 1000 built-in functions that can be called directly, from within a script, to perform a specific task.

Example:

Date(): returns the current system date

```
<?php
$d=date("D");
if ($d=="Sat")
echo "Have a nice Saturday!";
else
echo "Have a nice day!";
?>
```

PHP Functions

2. PHP User Defined Functions

- Besides the built-in PHP functions, it is possible to create your own functions.
  - A function is a block of statements that can be used repeatedly in a program.
  - A function will not execute automatically when a page loads.
  - A function will be executed by a call to the function.
- A user-defined function declaration starts with the word function:

Syntax

```
function functionName() {
    code to be executed;
}
```

- **Note:** A function name must start with a letter or an underscore. Function names are NOT case-sensitive

PHP Functions

Example

```
<?php
function writeMsg() {
    echo "Hello world!";
}
writeMsg(); // call the function
?>
```

User defined Function with Arguments

Information can be passed to functions through arguments. An argument is just like a variable. Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.

PHP Functions

The following example has a function with one argument ($fname).

Example

```
<?php
function familyName($fname) {
    echo "$fname.<br>";
}
familyName(“Yohans");
?>
```

The following example has a function with two arguments ($fname and $year):

Example

```
<?php
function familyName($fname, $year) {
    echo "$fname Nahom. Born in $year <br>";
}
familyName(“Alemu", "1975");
familyName(“Kebede", "1978");

?>
```

PHP Functions

PHP Functions - Returning values

To let a function return a value, use the return statement:

Example

```
<?php
function sum($x, $y) {
    $z = $x + $y;
    return $z;
}
echo "5 + 10 = " . sum(5, 10) . "<br>";
echo "7 + 13 = " . sum(7, 13) . "<br>";
echo "2 + 4 = " . sum(2, 4);
?>
```

OOP Concept

- Class
- Object
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation
- Package

Class

- A class is a blueprint or a template that defines a type of object.
- It encapsulates the properties (attributes) and behaviors (methods) that objects of that type will have.
- You define a class using the class keyword.
- Access modifiers define the visibility of class properties and methods
- In class, variables are called properties and functions are called methods!

Syntax:

```
<?php
class Fruit {
  // code goes here...
}
?>
```

Example

```
<?php
class Dog {

    public $name;
    public $breed;

    public function bark() {
        echo "Woof! Woof!";
    }
    public function fetch() {
        echo "{$this->name} is fetching.";
    }
}
?>
```

Object

- An object is an instance of a class.
- It is a concrete realization of the blueprint defined by the class.
- You can create multiple objects from the same class, each with its own set of property values.
- Objects of a class are created using the new keyword

Example

```php
<?php
// Creating objects from the Dog class
$dog1 = new Dog();
$dog2 = new Dog();
// Setting property values
$dog1->name = "Buddy";
$dog1->breed = "Labrador";
$dog2->name = "Max";
$dog2->breed = "Golden Retriever";
// Calling methods
$dog1->bark(); // Outputs: Woof! Woof!
$dog2->fetch(); // Outputs: Max is fetching.
?>
```

Abstraction

- It refers to the concept of hiding the complex implementation details of an object and exposing only the essential features or functionalities.
- It allows you to focus on what an object does rather than how it achieves its functionality.
- In PHP, abstraction is achieved through abstract classes and interfaces.

Example

```
// Abstract class representing a shape
abstract class Shape {
    // Abstract method for calculating the area
    abstract public function calculateArea();
}

class Circle extends Shape {
    private $radius;
    public function __construct($radius) {
        $this->radius = $radius;
    }
}
```

Cont…

```
class Rectangle extends Shape {
    private $length;
    private $width;
    public function __construct($length, $width) {
        $this->length = $length;
        $this->width = $width; }
    public function calculateArea() {
        return $this->length * $this->width;
    } }
$circle = new Circle(5);
$rectangle = new Rectangle(4, 6);
echo 'Circle Area: ' . $circle->calculateArea() . PHP_EOL;
echo‘ Rectangle Area:'.$rectangle->calculate Area(). PHP_EOL;
```

Encapsulation

- Encapsulation refers to the bundling of data and the methods that operate on that data into a single unit, often called a class.
- To hide the internal details of how an object works and expose only what is necessary for the outside world to interact with it.

Example

```
Class Car {
    // Private properties (attributes)
    private $model;
    private $color;
    private $fuelLevel;
    public function __construct($model, $color) {
        $this->model = $model;
        $this->color = $color;
        $this->fuelLevel = 100;}
    // Public method to get the model of the car
    public function getModel() {
        return $this->model;
}
```

Cont…

```
// Public method to get the color of the car
    public function getColor() {
        return $this->color;
    }
    // Public method to get the fuel level of the car
    public function getFuelLevel() {
        return $this->fuelLevel; }
// Public method to simulate driving, reducing fuel level
    public function drive() {
        // Simulate driving by reducing fuel level
        $this->fuelLevel -= 10;
        if ($this->fuelLevel < 0) {
            $this->fuelLevel = 0; // Ensure fuel level doesn't go below 0 }}}
```

Cont…

```
$myCar = new Car("Toyota", "Blue");
echo "Model: " . $myCar->getModel() . "<br>";
echo "Color: " . $myCar->getColor() . "<br>";
echo "Fuel Level: " . $myCar->getFuelLevel() . "%<br>";
$myCar->drive();
echo "Updated Fuel Level after driving: " . $myCar->getFuelLevel()
. "%";
```

Inheritance

- Inheritance is a concept where a new class (called the child or subclass) can inherit attributes and behaviors (properties and methods) from an existing class (called the parent or superclass).
- This allows you to create a relationship between classes.

Example

```
// Parent class
class Animal {
    public $name;
    public function __construct($name) {
        $this->name = $name; }
    public function eat() {
        return $this->name . ' is eating.'; }}
// Child class inheriting from Animal
class Dog extends Animal {
    public function bark() {
        return $this->name . ' is barking.';
    }}
```

Cont...

```
// Child class inheriting from Animal
class Cat extends Animal {
    public function meow() {
        return $this->name . ' is meowing.'; }}
// Create instances of the classes
$dog = new Dog('Buddy');
$cat = new Cat('Whiskers');
// Using inherited methods
echo $dog->eat(); // Output: Buddy is eating.
echo $dog->bark(); // Output: Buddy is barking.
echo $cat->eat(); // Output: Whiskers is eating.
echo $cat->meow(); // Output: Whiskers is meowing.
```

Polymorphism

- It allows objects of different classes to be treated as instances of the same class through a shared interface or base class.
- Method or function can behave differently based on the object it is called on, depending on the actual class of the object.
- To make your code more flexible, maintainable, and extensible.

Types of Polymorphism

There are two primary types of polymorphism:

I. Compile-Time Polymorphism (Static Polymorphism):

- ✓ This occurs when the method to be executed is determined at compile time.
- ✓ It is typically achieved through method overloading or operator overloading

Method Overloading:

- In method overloading, multiple methods can have the same name but differ in their arguments.
- In PHP, method overloading is achieved using the __call() magic method, because PHP does not support direct method overloading like other languages (e.g., Java).

Example: Using __call()

```
class Car {
    // Handle undefined method calls
    public function __call($method, $args) {
        echo "Method '$method' is called with arguments: " . implode(", ", $args) . "<br>";
    }
}
$car = new Car();
$car->drive(100, "fast");  // Outputs: Method 'drive' is called with arguments: 100, fast
$car->stop("immediately"); // Outputs: Method 'stop' is called with arguments: immediately
```

Types of Polymorphism

II. Run-Time Polymorphism (Dynamic Polymorphism):

- ✓ This occurs when the method to be executed is determined at run time.
- ✓ It is achieved through method overriding.

Method Overriding:

- Method overriding is the process by which a subclass provides a specific implementation of a method that is already defined in its superclass.
- In PHP, method overriding is done by redefining the method in the child class, while keeping the same signature (method name, parameters, and return type).

Example

```
// Parent class
class Animal {
    public function sound() {
        echo "Animal makes a sound<br>";
    }
}
// Child class 1
class Dog extends Animal {
    public function sound() {
        echo "Dog barks<br>";
    }
}
```

Cont…

```
// Child class 2
class Cat extends Animal {
    public function sound() {
        echo "Cat meows<br>";
    }
}
$dog = new Dog();
$cat = new Cat();
$dog->sound();  // Outputs: Dog barks
$cat->sound();  // Outputs: Cat meows
```

Session and Cookies

Introduction to stateful web application

- A stateful web application is one that keeps track of the user's data (state) over multiple requests.
- This is essential for functionalities like user authentication, shopping carts, and maintaining preferences.
- However, stateful web applications maintain some form of data between requests.
- PHP provides two main mechanisms to maintain state: Sessions and Cookies.
  - Cookies are stored on the client-side (in the browser).
  - Sessions are stored on the server-side, with a session identifier usually stored in the client's browser (commonly in cookies).

Cookie Attributes

- A cookie is a small piece of data stored on the client-side (in the browser) that can be sent to the server with every HTTP request.
- Cookies can be used to remember user preferences, login details, or shopping cart information.

Cookie attributes:

I. Name: The name of the cookie.

II. Value: The data associated with the cookie.

III. Expiration time (expires): The date/time when the cookie should expire. If no expiration time is set, the cookie will last for the current session (until the browser is closed).

Cont…

IV. Path (path): Defines the path on the server where the cookie is available. It limits the scope to certain paths.

V. Domain (domain): Defines the domain to which the cookie is available. For example, .example.com would allow the cookie to be accessible to www.example.com, api.example.com, etc.

VI. Secure: If set to true, the cookie will only be sent over HTTPS connections.

VII. HttpOnly: If set, the cookie can only be accessed via HTTP requests, and not through JavaScript (document.cookie), providing extra security.

Creating a Cookie

In PHP, you can create a cookie using the setcookie() function.

Syntax of setcookie() function:

setcookie(name, value, expiration, path, domain, secure, httponly);

Example:

setcookie("user", "JohnDoe", time() + 3600, "/"); // Cookie expires in 1 hour

// This sets a cookie named "user" with the value "JohnDoe", and it expires after 1 hour.

Retrieve Cookie Value

To retrieve the value of a cookie, you use the $_COOKIE superglobal array.

Example:

```
if (isset($_COOKIE["user"])) {
    echo "Hello, " . $_COOKIE["user"]; // Outputs: Hello, JohnDoe
} else {
    echo "Cookie not set.";
}
```

Delete Cookie

- To delete a cookie in PHP, you essentially set it with an expiration date in the past.
- This instructs the browser to remove the cookie.

Example:

```
// Delete a cookie
setcookie("username", "", time() - 3600, "/"); // The cookie will be deleted
```

Starting and Establishing a Session

- In PHP, you can start a session using the session_start() function. This function must be called at the beginning of your script before any output is sent to the browser.

Example:

```
<?php
 // Start the session
    session_start();
    // Set session variables
    $_SESSION['username'] = "JohnDoe";
    $_SESSION['role'] = "administrator";
    echo "Session variables are set.";
  ?>
```

Destroying a Session

To destroy a session, you need to first call session_start() to access session variables, then clear the session data and finally destroy the session itself.

Example:

```
<?php
// Start the session
session_start();
// Clearing all session variables
$_SESSION = array();
// If you want to delete the session cookie as well, uncomment the following line:
//
```

Cont…

```
if (ini_get("session.use_cookies")) {
//     $params = session_get_cookie_params();
//     setcookie(session_name(), '', time() - 42000,
//         $params["path"], $params["domain"],
//         $params["secure"], $params["httponly"]
//     );
// }
// Finally, destroy the session
session_destroy();
echo "Session destroyed.";
?>
```

Exception Handling

How exception handling works in PHP

- **Exceptions:** are objects that represent a runtime error or unexpected behavior in your code.
  - You can throw exceptions when a problem occurs.
- **Try-Catch Block:** encapsulate code that may throw an exception in a try block, and catch the exception in a catch block.
- **Throwing Exceptions:** use the throw statement to throw an exception manually.
- **Finally Block:** A finally block can be used after try and catch, which will execute regardless of whether an exception was thrown or not

Syntax

```
<?php
class CustomException extends Exception {
    public function errorMessage() {
        // error message
            return 'Error on line ' . $this->getLine() . ' in ' . $this-
>getFile() . ': ' . $this->getMessage();
    }
}
try {
    // Code that might throw an exception
    $value = 10;
    if ($value > 5) {
```

Cont…

```
 throw new CustomException('Value must not exceed 5.');
    }
} catch (CustomException $e) {
    // Handle the exception
    echo $e->errorMessage();
} catch (Exception $e) {
    // Handle any other type of exception
    echo 'Caught general Exception: ' . $e->getMessage();
} finally {
    // This block will execute regardless of the outcome
    echo 'Execution complete.';
}
?>
```

Thank You!!!